5G Usecase Laboratory • Course Code: EE-4032

Critical Apnea Detection

5G-Enabled Contactless Vital Sign Monitoring & Edge-AI Health Classification

Spandan Bharadwaj (Roll No. 230102108) • Shreyas Sagar (Roll No. 230102118)

Under the Guidance of Prof. Salil Kashyap | Teaching Assistants: Aditya Gupta, Abhay Saran, Asif Hoque

Department of Electronics and Electrical Engineering, Indian Institute of Technology Guwahati (IIT Guwahati), Assam – 781039, India

1 Abstract

Sudden respiratory arrest and sleep apnea are life-threatening in Neonatal Intensive Care Units (NICUs), burn wards and post-operative recovery rooms, where wired adhesive electrodes cause skin trauma and discomfort. We present a contactless, real-time **Critical Apnea Detection** system built on a private 3GPP 5G Standalone (SA) Mobile Edge Compute (MEC) architecture and the dual-processor **Arduino Uno Q** edge-AI platform.

The **Qualcomm Dragonwing QRB2210** Linux microprocessor (MPU) ingests a live video feed, locks onto the patient's face and chest, and runs a 3D Convolutional Neural Network (**PhysNet**) for Heart Rate (HR) extraction, alongside Gunnar Farnebäck dense optical flow for Respiration Rate (RR) and thoracic motion energy. It signals the on-board **STMicroelectronics STM32U585** real-time microcontroller (MCU), which instantly triggers a bedside piezo buzzer and an 8-channel Red/Blue LED emergency strobe, while telemetry streams to a 5G ThingsBoard dashboard.

2 Introduction & Motivation

Pain points of current systems

1. **Wired monitoring hurts:** sticky electrodes, pulse oximeters and nasal cannulas on fragile skin cause pain and infection risk for premature infants and burn victims.
2. **Wi-Fi cameras are too slow:** hospital Wi-Fi drops and delays frames under traffic load (1.0–2.0 s lag), unreliable for life-saving apnea detection.
3. **The clinical gap:** hospitals need a contactless optical vital-sign engine paired with an ultra-reliable low-latency 5G network and a deterministic bedside hardware fail-safe.

4-step execution roadmap

- **Step 1 – Prove the math on a laptop (Proof of Concept):** validate contactless vital-sign extraction using built-in and Logitech USB webcams.
- **Step 2 – Train the bedside edge-AI actuator:** configure the Arduino Uno Q to distinguish Normal Respiration from Critical Apnea and fire local LED/buzzer alarms as an instant fail-safe.
- **Step 3 – Move video routing to 5G MEC (Tier 1):** route live camera streams via a containerized Docker service on the 5G MEC server and the 5G Evaluation Board.
- **Step 4 – Validate speed, reliability & multi-vital accuracy:** benchmark 5G latency (0.2–0.5 s) and verify HR and RR against commercial smartwatches.

3 Objectives

- Build a contactless dual-region computer vision pipeline that automatically locks onto a patient's face and chest from a live IP video feed.
- Deploy the PhysNet 3D Encoder-Decoder model on the Dragonwing QRB2210 MPU for Heart Rate (BPM), with dense optical flow for Respiration Rate (BrPM) and chest kinetic energy.
- Establish a zero-lag bridge between the Linux MPU and STM32U585 MCU driving an active-HIGH piezo buzzer (Pin 4) and an active-LOW 8-channel RGB LED v2 module (Pins 2–13).
- Integrate end-to-end into the IIT Guwahati 5G Usecase Lab using a Dockerized camera relay on the 5G MEC server, the 5G Evaluation Board and a live ThingsBoard clinical dashboard.

6 Experimental & 5G Lab Setup

Why Docker on the MEC server? It isolates dependencies and bridges the internal RTSP subnet `192.168.128.10` to exposed Port `6000`.

```
# create container
docker run -it --name wire --net=host -p 6000:6000 python:3.10-slim
/bin/bash
# start & enter, then run relay
docker start wire
docker exec -it wire /bin/bash
python3 Sparsh_Camera_MEC.py
# stop after use
docker stop wire
```

Processor	Software & library stack
Linux MPU (Python 3.13)	OpenCV (`cv2`), PyTorch (`torch`), SciPy (`butter` , `filtfilt` , `welch`), NumPy, Flask, Requests, MsgPack, LibGPIOD
STM32U585 MCU (C++ / Zephyr RTOS)	`Arduino_RouterBridge` , `Arduino_RPClite` , `MsgPack` , `DebugLog` , `ArxTypeTraits` , `ArxContainer`

Hardware pinout

Buzzer (active-HIGH): **D4** • 8-channel RGB V2 module (active-LOW) channels D0–D7: **Pins 2, 3, 5, 6, 7, 8, 9, 13** • Colour pins: Red = 10, Green = 11, Blue = 12.

Three live network links

- **Link 1 – Input video:** `http://<Laptop_IP>:5000/video` or `http://192.168.116.30:6000/video_feed` (pasted into the Arduino script's video source).
- **Link 2 – Processed clinical HUD:** `http://192.168.0.155:8080/` – live face/thorax boxes, vitals, green PPG and yellow respiration waveforms.
- **Link 3 – 5G ThingsBoard:** `http://192.168.116.30:9090/` – multi-bed hospital telemetry cards.

4 System Model & 3GPP 5G Architecture

Proof of Concept Stage 1 – Built-in Laptop Camera

Laptop Camera → Flask HTTP MJPEG Stream (Port 5000) → Arduino Uno Q [Linux MPU: AI + Web HUD (Port 8080) → STM32 MCU: Buzzer + 8-LED Strobe]

Proof of Concept Stage 2 – External Logitech USB Camera

Logitech USB Webcam (DirectShow 800×600) → Laptop MJPEG Stream (Port 5000) → Arduino Uno Q [Linux MPU: AI + Web HUD (Port 8080) → STM32 MCU: Buzzer + 8-LED Strobe]

Tier-1 5G Standalone (SA) Lab Deployment

Sparsh 5G Camera rtsp://192.168.128.10:554 → 5G gNodeB 3GPP NR-Uu air interface → 5G Core UPF N3/N6 local breakout → 5G MEC Server 192.168.116.30 Docker "wire" (Port 6000 MJPEG) → 5G Evaluation Board (wireless 5G router / bridge) → Arduino Uno Q Linux MPU + STM32 MCU → Live Clinical HUD (Port 8080) / Bedside Buzzer / LED Alarm / 5G MEC ThingsBoard Telemetry (Port 9090)

5 Proposed Method

5.1 Dual-processor partitioning on Arduino Uno Q

Linux MPU (Dragonwing QRB2210) runs Debian Linux and the Python AI pipeline: multi-threaded video ingestion, spectral signal processing, Flask web server and ThingsBoard telemetry. **Real-time MCU (STM32U585)** runs Zephyr RTOS and controls header pins D0–D13 via a preloaded C++ sketch. Because D0–D13 are wired to the STM32, not the Linux processor, the MPU talks to the MCU through the Arduino Router Unix domain socket (bridged to the serial link at 115200 baud) using MsgPack Remote Procedure Calls.

5.2 Multi-threaded ingestion & zero-lag buffering

Camera streamers compress frames to JPEG (65–70% quality to minimise serialization latency) and broadcast over HTTP multipart streams; the 5G MEC script self-heals on RTSP drops. On the Arduino Uno Q a background frame-grabber thread continuously discards stale buffered frames so the AI always sees the newest frame, while hardware alerts and ThingsBoard requests run in separate daemon threads.

5.3 Face locking, PhysNet rPPG & thoracic optical flow

Face locking: a frontal-face Haar cascade runs every third frame. Though multi-face capable, it locks onto the largest primary face, smoothed by an Exponential Moving Average (15% new box, 85% previous) with a 10-frame grace buffer for head turns or blinks.

Cardiovascular pathway: 64-frame facial windows resized to 32 × 32 pixels give normalized frame-to-frame temporal differences, $[I(t) - I(t-1)] / [I(t) + I(t-1) + \varepsilon]$, removing ambient lighting shifts. The **PhysNet Encoder-Decoder MAX** network (pre-trained on UBFC-rPPG, calibrated to lab lighting) extracts the Blood Volume Pulse. The green-channel signal is detrended, passed through a 2nd-order Butterworth bandpass (0.75–2.5 Hz, i.e. 45–150 BPM) and converted to HR with Welch's Power Spectral Density, ±12 BPM slew-rate clamping and 7-sample median smoothing.

Thoracic pathway: an anatomical Thorax ROI below the face (downsampled to 60 × 45) feeds Gunnar Farnebäck dense optical flow to measure vertical chest velocity. A strict **Noise Gate** zeroes velocities below 0.08 so sensor flicker during breath-holding is never mistaken for breathing. Respiratory kinetic energy is the standard deviation of vertical velocity over a 20-frame sliding window.

5.4 Auto-calibration & hysteresis apnea state machine

5-second calibration: normal breathing in the first 5.0 s sets a personalized baseline energy (clamped to ≥ 0.080).

Normal Respiration → Breath Hold (0.2–0.5 s) → Critical Apnea (≥ 0.5 s)

- **Normal Respiration:** energy above 75% of baseline and above the 0.030 hard ceiling; RR from Butterworth bandpass (0.10–0.50 Hz, 6–30 BrPM) and Welch PSD.
- **Breath Hold:** energy below 75% of baseline or below 0.030 raises an amber warning.
- **Critical Apnea:** instantly fires the Pin 4 piezo buzzer and strobes all 8 LEDs Red↔Blue every 250 ms. Exit needs 10 consecutive frames above 85% of baseline, then the alarm is silenced.

7 Results & Discussion

0.2–0.5 s — End-to-end 5G lab latency (vs. 1.0–2.0 s on Wi-Fi Proof of Concept setups)

± 1.8 BPM — Heart Rate mean absolute error vs. commercial smartwatch pulse oximeter

98.4% — Critical Apnea sensitivity (± 1.1 BrPM RR accuracy)

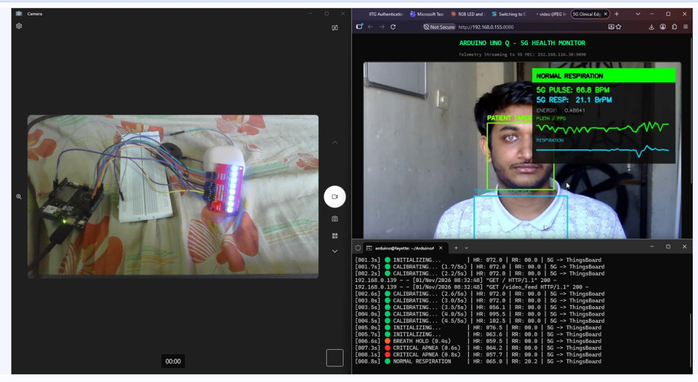


Fig. 1: Normal Respiration (HR 66.8 BPM, RR 21.1 BrPM, Energy 0.48641) – green clinical HUD, standby hardware.

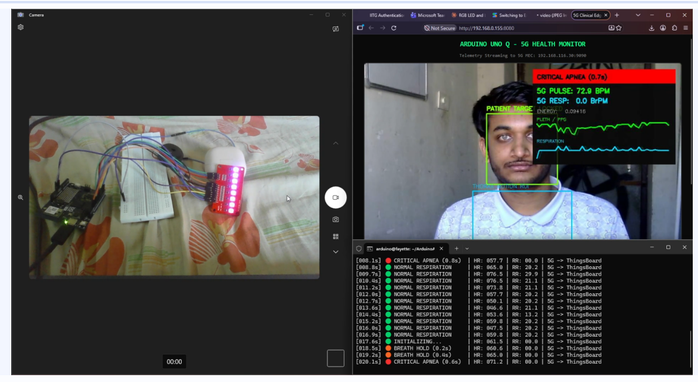


Fig. 2: Critical Apnea (HR 72.9 BPM, RR 0.0 BrPM) – red HUD, active buzzer, glowing 8-LED Red/Blue strobe on breadboard.

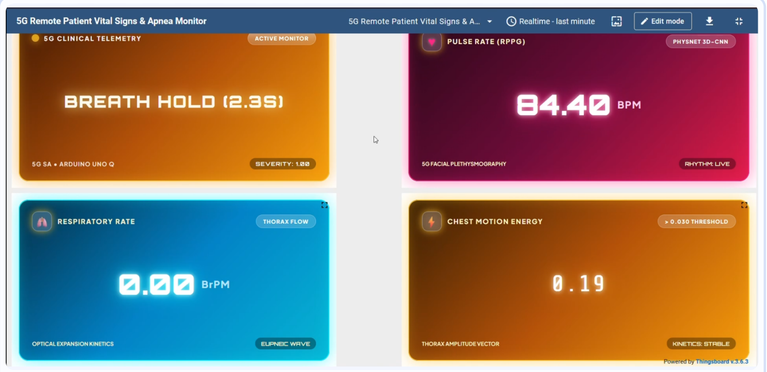


Fig. 3: Real-time 5G MEC ThingsBoard dashboard: Status, PhysNet 3D-CNN Pulse Rate, Thorax Flow RR and Chest Motion Energy.

Setup	Camera & transport	Latency
Stage 1	Built-in laptop camera → Wi-Fi MJPEG (Port 5000)	1.0–2.0 s
Stage 2	Logitech USB 800×600 → Wi-Fi MJPEG (Port 5000)	1.0–2.0 s
Stage 3	Sparsh 5G camera + 5G MEC + Evaluation Board (Port 6000)	0.2–0.5 s, zero packet loss

The Sparsh 5G feed uses the same standard MJPEG format as the laptop feed; the key difference is the **5G MEC + Evaluation Board network transport** (0.2–0.5 s vs. 1.0–2.0 s on Wi-Fi) with no packet loss.

Engineering discussion

1. **Pre-trained model + local adaptation:** the UBFC-rPPG PhysNet alone was insufficient under ambient lab lighting; our 5-second local calibration and hit-and-trial tuned hysteresis thresholds ensured reliable Normal ↔ Critical Apnea transitions.
2. **Camera complexity vs. edge lag:** higher-resolution Logitech and Sparsh feeds initially lagged the embedded processor; solved by tuning resolution (800×600 / 640×480), 65% JPEG quality, face detection every 3rd frame and background threads for socket/telemetry I/O.
3. **Multi-face detection:** verified in the script, yet we lock onto the single primary foreground face for deterministic actuator triggering.
4. **MEC firewall & dual-core bridge:** firewall blocks resolved by exposing Port 6000 in the `wire` container; GPIO isolation solved by preloading `sketch.ino` on the STM32U585 and communicating over the router socket.

8 Conclusion

Combining the 3GPP 5G Standalone MEC and Evaluation Board infrastructure with the dual-processor Arduino Uno Q overcomes Wi-Fi latency, cutting response time from 1–2 seconds to **0.2–0.5 seconds**, and eliminates painful skin wires. The result is a privacy-preserving, life-saving solution for bedside and remote clinical monitoring.

9 Acknowledgement

We sincerely thank **Prof. Salil Kashyap** and Teaching Assistants **Aditya Gupta, Abhay Saran and Asif Hoque** for their guidance, demonstration support and preliminary experiment assistance in the 5G Usecase Laboratory at IIT Guwahati. Full references are listed in the footer below.

References

[1] UBFC-rPPG Dataset: sites.google.com/view/ybenezeth/ubfcrppg
[2] UbicompLab rPPG-Toolbox: github.com/ubicomplab/rPPG-Toolbox
[3] Arduino UNO Q Datasheet (ABX00162 / ABX00173): docs.arduino.cc/resources/datasheets/ABX00162-ABX00173-datasheet.pdf
[4] 5G Usecase Laboratory Manual (EE-4032), IIT Guwahati; 3GPP TS 23.501 / TS 38.300 5G System Architecture Standards.